

Reducing Circuit SAT to Maximum Independent Set

Notes on NP-Completeness Reductions

Abstract

This note presents the polynomial-time reduction from Circuit SAT to the Maximum Independent Set (MIS) problem, establishing that MIS is NP-complete. The reduction proceeds in two stages: first from Circuit SAT to 3-SAT via the Tseytin transformation, then from 3-SAT to MIS using clause gadgets. We provide detailed constructions with illustrative TikZ diagrams.

1 Introduction to NP-Completeness

Computational complexity theory classifies problems by the resources required to solve them. A central concept in this field is **NP-completeness**, which identifies the “hardest” problems in the class NP.

Definition 1 (The Class P). *P is the class of decision problems solvable by a deterministic Turing machine in polynomial time. These are problems we consider “efficiently solvable.”*

Definition 2 (The Class NP). *NP (nondeterministic polynomial time) is the class of decision problems for which a “yes” answer can be verified in polynomial time, given an appropriate certificate (or witness). Equivalently, NP is the class of problems solvable by a nondeterministic Turing machine in polynomial time.*

Definition 3 (Polynomial-Time Reduction). *A problem A is polynomial-time reducible to problem B, written $A \leq_p B$, if there exists a polynomial-time computable function f such that for all instances x :*

$$x \in A \iff f(x) \in B$$

Intuitively, if we can solve B efficiently, then we can solve A efficiently by first transforming the input.

Definition 4 (NP-Complete). *A problem B is **NP-complete** if:*

- 1. $B \in \text{NP}$ (solutions can be verified in polynomial time), and*
- 2. Every problem $A \in \text{NP}$ satisfies $A \leq_p B$ (B is NP-hard).*

NP-complete problems are significant because they represent a class of problems that are all polynomially equivalent—if any one of them admits a polynomial-time algorithm, then *all* problems in NP can be solved in polynomial time (implying $P = \text{NP}$). The famous Cook-Levin theorem established that Circuit SAT (and Boolean SAT) are NP-complete, providing the foundation for proving other problems NP-complete through polynomial-time reductions.

In this note, we demonstrate that the Maximum Independent Set problem is NP-complete by reducing from Circuit SAT, which is known to be NP-complete.

2 Problem Definitions

Definition 5 (Circuit SAT). *Given a Boolean circuit C with n input variables $x_1, \dots, x_n$ and a single output, determine whether there exists an assignment to the input variables such that C outputs **TRUE**.*

Definition 6 (Maximum Independent Set (MIS)). *Given an undirected graph $G = (V, E)$ and an integer k , determine whether there exists a subset $S \subseteq V$ with $|S| \geq k$ such that no two vertices in S are adjacent.*

Definition 7 (3-SAT). *Given a Boolean formula ϕ in conjunctive normal form (CNF) where each clause contains exactly three literals, determine whether there exists a satisfying assignment.*

3 Overview of the Reduction Chain

The reduction from Circuit SAT to MIS proceeds in two steps:

$$\text{Circuit-SAT} \leq_p \text{3-SAT} \leq_p \text{Independent Set}$$



Figure 1: The reduction chain from Circuit SAT to Maximum Independent Set

4 Step 1: Circuit SAT to 3-SAT (Tseytin Transformation)

The Tseytin transformation converts a Boolean circuit into an equisatisfiable CNF formula with size linear in the circuit size.

4.1 Key Idea

For each gate in the circuit, we introduce a new variable representing its output and add clauses that enforce the gate's behavior. This avoids the exponential blowup that can occur with naive conversion.

4.2 Gate Encodings

For each type of gate, we create CNF clauses that are satisfiable if and only if the output correctly represents the gate's function:

AND Gate: For $C = A \wedge B$:

$$(\neg A \vee \neg B \vee C) \wedge (A \vee \neg C) \wedge (B \vee \neg C)$$

OR Gate: For $C = A \vee B$:

$$(A \vee B \vee \neg C) \wedge (\neg A \vee C) \wedge (\neg B \vee C)$$

NOT Gate: For $C = \neg A$:

$$(\neg A \vee \neg C) \wedge (A \vee C)$$

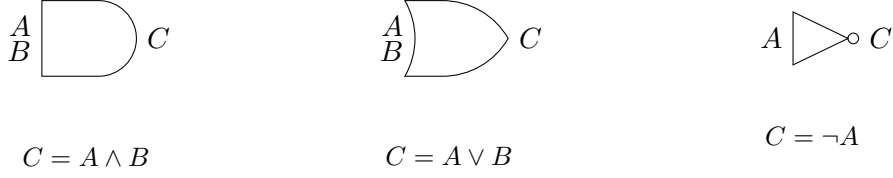


Figure 2: Basic logic gates used in circuits

4.3 Complete Example

Consider the circuit computing $f(x_1, x_2, x_3) = (x_1 \wedge x_2) \vee x_3$:

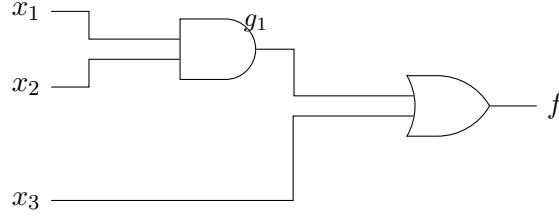


Figure 3: Circuit for $f(x_1, x_2, x_3) = (x_1 \wedge x_2) \vee x_3$

The Tseytin transformation produces:

$$\begin{aligned}
 \phi = & (\neg x_1 \vee \neg x_2 \vee g_1) \wedge (x_1 \vee \neg g_1) \wedge (x_2 \vee \neg g_1) && \text{(AND gate)} \\
 \wedge & (g_1 \vee x_3 \vee \neg f) \wedge (\neg g_1 \vee f) \wedge (\neg x_3 \vee f) && \text{(OR gate)} \\
 \wedge & (f) && \text{(output must be TRUE)}
 \end{aligned}$$

Theorem 1. *The Tseytin transformation runs in polynomial time and produces a formula ϕ such that:*

1. $|\phi| = O(|C|)$ where $|C|$ is the size of the circuit
2. The circuit C is satisfiable if and only if ϕ is satisfiable

5 Step 2: 3-SAT to Maximum Independent Set

5.1 Construction

Given a 3-SAT formula ϕ with m clauses $C_1, \dots, C_m$, we construct a graph G_ϕ and set $k = m$.

Vertex Construction: For each clause $C_j = (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$, create three vertices $v_{j,1}, v_{j,2}, v_{j,3}$, one for each literal.

Edge Construction: Add two types of edges:

1. **Clause edges (triangles):** For each clause C_j , connect all three vertices in a triangle:

$$\{v_{j,1}, v_{j,2}\}, \{v_{j,2}, v_{j,3}\}, \{v_{j,1}, v_{j,3}\}$$

2. **Conflict edges:** For any two vertices $v_{j,a}$ and $v_{k,b}$ (from different clauses) whose labels are complementary literals (i.e., one is x_i and the other is $\neg x_i$), add an edge.

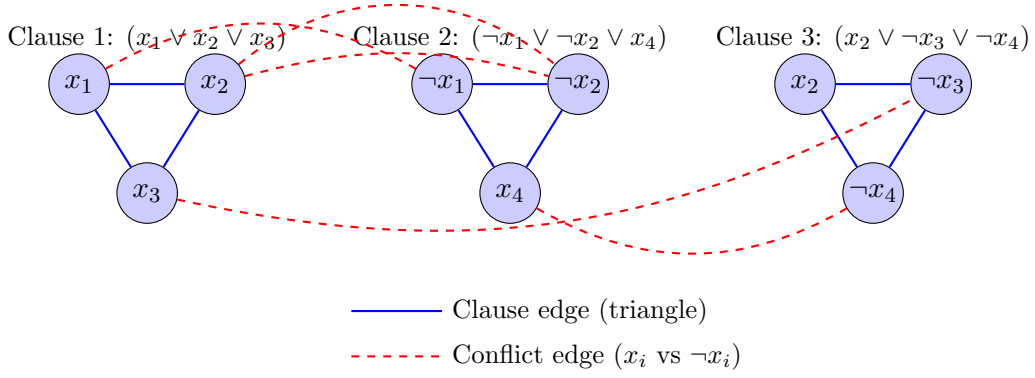


Figure 4: Graph construction for the formula $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee \neg x_3 \vee \neg x_4)$

5.2 Intuition Behind the Construction

- The **triangle structure** ensures we can select at most one vertex per clause (choosing the “true” literal).
- The **conflict edges** ensure we cannot simultaneously select x_i from one clause and $\neg x_i$ from another (consistency).
- An independent set of size m corresponds to selecting exactly one true literal per clause without contradictions.

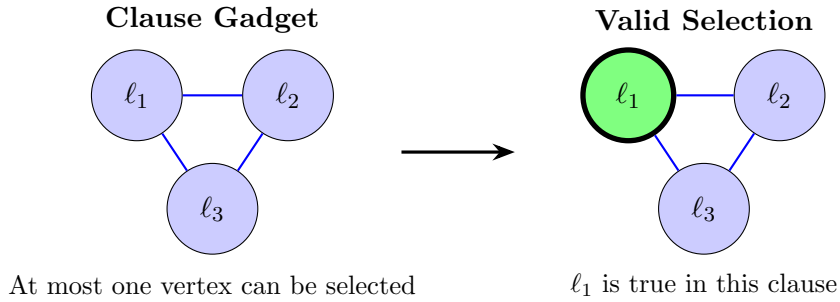


Figure 5: The clause gadget enforces selecting at most one “true” literal per clause

5.3 Proof of Correctness

Lemma 2. ϕ is satisfiable if and only if G_ϕ has an independent set of size m .

Proof. ($\Rightarrow$) Suppose ϕ is satisfiable with assignment α . For each clause C_j , at least one literal is true under α . Select one such true literal’s vertex from each clause. This gives exactly m vertices.

Claim: These vertices form an independent set.

- No two selected vertices are from the same clause (we pick exactly one per clause), so no clause edges connect them.
- No two selected vertices have complementary labels (since α is consistent—it cannot set both x_i and $\neg x_i$ to true), so no conflict edges connect them.

($\Leftarrow$) Suppose G_ϕ has an independent set S of size m . Since each clause contributes a triangle and S can contain at most one vertex from each triangle, and $|S| = m$, S must contain exactly one vertex from each clause.

Define assignment α : for each selected vertex labeled ℓ :

- If $\ell = x_i$, set $x_i = \mathbf{true}$
- If $\ell = \neg x_i$, set $x_i = \mathbf{false}$

Claim: α is well-defined and satisfies ϕ .

- Well-defined: If S contained both x_i and $\neg x_i$, they would be connected by a conflict edge, contradicting independence.
- Satisfies ϕ : Each clause has a selected vertex, whose label is a true literal under α .

□

6 Complexity Analysis

Theorem 3. *The reduction from Circuit SAT to Maximum Independent Set runs in polynomial time.*

Proof. **Step 1 (Tseytin):**

- Each gate produces $O(1)$ clauses with $O(1)$ literals
- Total: $O(|C|)$ clauses where $|C|$ is the circuit size

Step 2 (3-SAT to MIS):

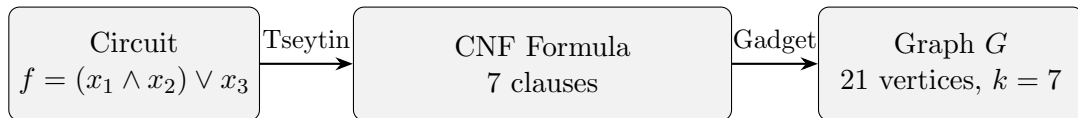
- $3m$ vertices (3 per clause)
- $O(m^2)$ edges (triangles plus conflict edges)
- Construction time: $O(m^2)$

Combined: $O(|C|^2)$ total time.

□

7 Complete Example

Consider the circuit and formula from our running example. Here is the complete reduction visualized:



8 Summary

Key Points:

1. Circuit SAT captures all of NP (Cook-Levin theorem)
2. Tseytin transformation: each gate $\rightarrow$ small CNF, linear blowup
3. 3-SAT to MIS: clause triangles + conflict edges
4. Both reductions are polynomial-time
5. Therefore, Maximum Independent Set is NP-complete

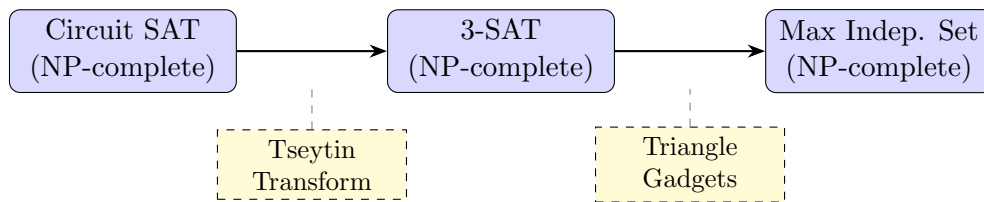


Figure 6: Summary of the reduction chain

References

1. Tseytin, G. S. (1983). On the complexity of derivation in propositional calculus.
2. Karp, R. M. (1972). Reducibility among combinatorial problems.
3. Sipser, M. (2012). Introduction to the Theory of Computation. Cengage Learning.